ARQUITETURA CLOUD-NATIVE PARA SISTEMAS ESCALÁVEIS: INTEGRAÇÃO DE MICROSSERVIÇOS, SERVERLESS E OBSERVABILIDADE

CLOUD-NATIVE ARCHITECTURE FOR SCALABLE SYSTEMS: INTEGRATING MICROSERVICES, SERVERLESS AND OBSERVABILITY

Gabriel Ribeiro de Camargo Miranda[1]; Deivison Shindi Takatu[2]; Glauco Todesco[3]

[1] Discente de Engenharia de Software. [2],[3] Orientadores da pesquisa. As afiliações e os e-mails institucionais devem ser conferidos antes do cadastro na plataforma do evento.

RESUMO

O crescimento contínuo da demanda por aplicações digitais exige soluções capazes de evoluir com segurança, disponibilidade e controle de custos. Este trabalho apresenta uma proposta de arquitetura cloud-native para sistemas escaláveis, elaborada no contexto de uma pesquisa de Iniciação Científica em desenvolvimento. A proposta integra front-end desacoplado, API Gateway, autenticação centralizada, funções serverless, microsserviços em contêineres, persistência híbrida e observabilidade. A pesquisa adota abordagem aplicada, exploratória e qualitativa, combinando levantamento bibliográfico, análise de requisitos não funcionais e modelagem arquitetural. Como resultados parciais, foram produzidos uma descrição técnica da solução, diagramas dos componentes e dos fluxos de comunicação, uma matriz de decisão tecnológica e uma organização de repositório para rastreabilidade do trabalho. A análise inicial indica que a combinação de serviços gerenciados, comunicação assíncrona e escalabilidade horizontal pode reduzir o acoplamento e facilitar a evolução de sistemas, mas também introduz desafios de segurança, monitoramento e governança. Não são apresentados resultados de desempenho medidos, pois a etapa de prototipagem e testes de carga ainda não foi concluída. Conclui-se preliminarmente que a arquitetura proposta é tecnicamente coerente com os requisitos investigados e constitui base para validação experimental posterior.

Palavras-chave: arquitetura cloud-native; microsserviços; computação serverless; escalabilidade; observabilidade.

ABSTRACT

The continuous growth in demand for digital applications requires solutions capable of evolving with security, availability, and cost control. This paper presents a cloud-native architecture proposal for scalable systems, developed within an ongoing Undergraduate Scientific Research project. The proposal integrates a decoupled front end, an API Gateway, centralized authentication, serverless functions, containerized microservices, hybrid persistence, and observability. The research adopts an applied, exploratory, and qualitative approach, combining a literature review, analysis of non-functional requirements, and architectural modeling. As partial results, a technical description of the solution, component and communication-flow diagrams, a technology decision matrix, and a repository structure for research traceability were produced. The initial analysis indicates that the combination of managed services,

asynchronous communication, and horizontal scalability can reduce coupling and facilitate system evolution, while also introducing challenges related to security, monitoring, and governance. No measured performance results are presented because the prototyping and load-testing stages have not yet been completed. The preliminary conclusion is that the proposed architecture is technically consistent with the investigated requirements and provides a basis for subsequent experimental validation.

Keywords: cloud-native architecture; microservices; serverless computing; scalability; observability.

1 INTRODUÇÃO

A transformação digital ampliou a necessidade de sistemas capazes de suportar variações de carga, integração com serviços externos e evolução frequente. Aplicações desenvolvidas como monólitos podem inicialmente reduzir a complexidade de implantação, mas tendem a concentrar responsabilidades e dificultar a evolução independente de módulos. Em ambientes de nuvem, simplesmente transferir esse modelo para uma infraestrutura remota não garante elasticidade, confiabilidade ou eficiência operacional. A arquitetura precisa relacionar os requisitos do sistema às capacidades da plataforma e às práticas de engenharia de software [1][2].

Este trabalho consolida uma proposta de arquitetura cloud-native para sistemas escaláveis, com foco na integração entre microsserviços, computação serverless e observabilidade. A pesquisa está em desenvolvimento e não pretende apresentar resultados experimentais que ainda não foram coletados. O objetivo desta etapa é organizar o problema, fundamentar as decisões técnicas e preparar um artefato arquitetural que possa ser validado posteriormente por meio de protótipo e testes de carga.

1.1 Problema de pesquisa

Como estruturar uma arquitetura de software que permita a evolução e a escalabilidade de sistemas em nuvem, reduzindo o acoplamento entre componentes sem transferir a complexidade para a operação, a segurança e o monitoramento?

1.2 Objetivo

O objetivo geral é propor uma arquitetura cloud-native para sistemas escaláveis, combinando boas práticas de engenharia de software com serviços gerenciados de computação em nuvem. Como objetivos específicos, pretende-se: identificar requisitos não funcionais relevantes; analisar padrões de microsserviços, serverless e comunicação assíncrona; definir componentes e responsabilidades; especificar mecanismos de segurança e observabilidade; e organizar um plano de validação por protótipo e testes de carga.

1.3 Justificativa

A relevância do estudo decorre da necessidade de orientar decisões arquiteturais em cenários nos quais a adoção de tecnologias de nuvem ocorre sem uma análise sistemática de trade-offs. A literatura aponta benefícios de modularidade e escalabilidade em microsserviços, mas também evidencia custos relacionados à comunicação distribuída, à consistência dos dados e à operação de múltiplos serviços [2]. Uma proposta documentada e rastreável pode apoiar o aprendizado acadêmico e servir como referência para a implementação gradual de sistemas com requisitos de crescimento e disponibilidade.

2 REVISÃO DE LITERATURA

Microsserviços são uma abordagem para decompor uma aplicação em serviços independentes, orientados a capacidades de negócio e com possibilidade de implantação autônoma. Newman [1] destaca que a definição das fronteiras deve considerar o domínio e o contexto da organização, evitando a criação de um monólito distribuído. O estudo de mapeamento sistemático de Cerny et al. [2] identifica como benefícios recorrentes a evolução independente, a escalabilidade seletiva e a autonomia das equipes, mas registra desafios relacionados à integração, aos testes e à observabilidade.

A computação serverless desloca para o provedor parte significativa da administração da infraestrutura. Revisões sobre o paradigma mostram que funções sob demanda são adequadas a eventos, APIs e cargas variáveis, especialmente quando o tempo de execução é limitado e a elasticidade é prioritária [3][4]. Estudos de adoção industrial também discutem impactos econômicos e arquiteturais desse modelo [15]. Entretanto, limitações como inicialização a frio, restrições de execução, dependência do provedor e dificuldade de depuração distribuída precisam ser consideradas antes da escolha tecnológica [5][6]. Por esse motivo, este trabalho não propõe serverless como solução universal, mas como uma das estratégias de processamento.

A combinação de funções e contêineres permite adequar o modelo de execução ao perfil de cada componente. O uso de contêineres favorece portabilidade e controle sobre dependências, enquanto funções serverless simplificam rotinas reativas e de curta duração [7]. A comunicação assíncrona por filas e tópicos pode reduzir a dependência temporal entre produtor e consumidor, absorver picos de demanda e melhorar a resiliência, desde que sejam tratados idempotência, reprocessamento e mensagens não processáveis.

Segurança e observabilidade são requisitos estruturais, e não complementos adicionados ao final do desenvolvimento. Em microsserviços, autenticação, autorização, criptografia e isolamento de rede precisam ser aplicados de modo consistente entre os serviços [8]. O AWS Well-Architected Framework recomenda avaliar segurança, confiabilidade, excelência operacional, eficiência de performance e otimização de custos como dimensões relacionadas da solução [9]. A observabilidade, apoiada por logs estruturados, métricas e rastreamento distribuído, permite verificar se os requisitos estão sendo atendidos e localizar falhas em fluxos que atravessam múltiplos componentes.

3 METODOLOGIA

Esta pesquisa é aplicada, exploratória e de abordagem qualitativa. O objeto de estudo é uma proposta arquitetural para sistemas escaláveis em nuvem. O trabalho foi organizado em ciclos de investigação e documentação, combinando revisão bibliográfica, análise de requisitos e modelagem de software.

Inicialmente, foram selecionadas referências sobre microsserviços, serverless, segurança e arquitetura em nuvem. Em seguida, os problemas observados em aplicações monolíticas foram relacionados a requisitos não funcionais: escalabilidade horizontal, disponibilidade, manutenibilidade, segurança, observabilidade e controle de custos. A partir dessa relação, foi elaborada uma arquitetura em camadas, com front-end desacoplado, API Gateway, autenticação, processamento síncrono e assíncrono, persistência híbrida e monitoramento. A organização da investigação como estudo aplicado e conceitual dialoga com trabalhos recentes desenvolvidos no contexto do ensino de Engenharia de Software [14].

A modelagem foi documentada em diagramas Mermaid e em uma descrição técnica dos componentes. As decisões foram registradas por justificativas, riscos e alternativas, permitindo rastrear a relação entre problema, requisito e tecnologia. A validação realizada nesta fase é documental e por inspeção de coerência. A validação experimental, com protótipo funcional, testes de carga e coleta de métricas como latência, taxa de erro e throughput, permanece como etapa futura.

4 RESULTADOS E DISCUSSÕES

Os resultados parciais consistem em um artefato arquitetural consolidado e na organização dos materiais de pesquisa. A camada de apresentação utiliza uma aplicação web desacoplada, distribuída por Amazon CloudFront e armazenamento de objetos. O API Gateway funciona como ponto de entrada para as chamadas da aplicação e integra autenticação e controle de tráfego. Essa separação reduz a dependência direta do cliente em relação à implementação interna dos serviços [9][10] [13].

Na camada de processamento, a solução combina funções AWS Lambda [11] e microsserviços em ECS/Fargate. Funções de curta duração, integrações e consumidores de eventos podem ser implementados de forma serverless. Serviços que exigem maior controle de runtime, processamento prolongado ou dependências específicas podem ser executados em contêineres. Essa decisão híbrida é um resultado importante da análise, pois reconhece que a arquitetura deve se adaptar ao perfil da carga de trabalho, e não o contrário [3][7].

Na persistência, o RDS PostgreSQL é destinado a dados relacionais e transações que exigem consistência, enquanto o DynamoDB é indicado para dados semiestruturados, sessões e fluxos de alta variabilidade. Filas e tópicos SQS/SNS são empregados para desacoplar processamento assíncrono. A proposta exige que cada serviço seja responsável por suas próprias regras de acesso aos dados e que os eventos tenham identificadores idempotentes. Essas restrições são necessárias para evitar inconsistências e efeitos duplicados durante retentativas.

Em segurança, foram definidos autenticação centralizada com Amazon Cognito [12], autorização baseada em tokens, criptografia em trânsito, armazenamento protegido e sub-redes privadas para os componentes de dados. Em observabilidade, foram definidos logs estruturados, métricas de operação, alertas e rastreamento distribuído. A arquitetura também prevê escalabilidade horizontal, redundância entre zonas de disponibilidade, retentativas com backoff e limites de concorrência. O Quadro 1 explicita a relação entre requisitos, decisões tecnológicas e a validação prevista.

Quadro 1 - Matriz de decisão tecnológica da arquitetura proposta

Requisito	Decisão arquitetural	Justificativa e validação prevista
Escalabilidade	Lambda e ECS/Fargate com expansão horizontal	Adequar ao perfil da carga; validar latência, erros e throughput.
Segurança	Cognito, tokens, TLS, IAM e sub-redes privadas	Aplicar autenticação, autorização e isolamento; validar configuração e ameaças.
Persistência	RDS PostgreSQL e DynamoDB	Separar dados transacionais e semiestruturados; validar consistência e concorrência.
Observabilidade	CloudWatch, logs, métricas e rastreamento	Detectar falhas e acompanhar indicadores; validar com injeção de falhas.
Resiliência	SQS/SNS, retentativas, backoff e idempotência	Desacoplar consumidores; validar reprocessamento e mensagens não processáveis.

Os resultados apresentados são decisões de projeto, não medições de desempenho; portanto, sua efetividade ainda precisa ser avaliada empiricamente.

As discussões realizadas até o momento evidenciam um trade-off central: a arquitetura distribuída aumenta a capacidade de escalar componentes de forma independente, mas amplia o número de pontos de falha e a necessidade de governança. A adoção de serviços gerenciados pode reduzir o trabalho de infraestrutura, porém cria dependências de configuração, custos variáveis e possível acoplamento ao provedor. Dessa forma, a solução deve ser validada com cenários controlados antes de uma eventual implementação produtiva.

5 CONCLUSÃO

Este trabalho apresentou uma proposta inicial de arquitetura cloud-native para sistemas escaláveis, relacionando microsserviços, serverless, persistência híbrida, segurança e observabilidade. O objetivo da etapa foi alcançado por meio da definição do problema, da revisão de literatura, da modelagem dos componentes e do registro das principais decisões técnicas.

Como conclusão preliminar, a combinação de modelos de execução e serviços gerenciados mostra-se adequada para tratar diferentes perfis de carga e reduzir o acoplamento entre camadas. Contudo, a proposta não permite afirmar ganhos quantitativos de desempenho ou custo, pois ainda não foram realizados protótipo e testes experimentais. Como próximos passos, recomenda-se implementar uma prova de conceito, estabelecer cenários de carga, medir latência e taxa de erro, avaliar falhas parciais e revisar os resultados com os orientadores. A pesquisa permanece, portanto, em desenvolvimento e aberta a ajustes após a avaliação acadêmica.

REFERÊNCIAS

[1] NEWMAN, S. *Building Microservices: Designing Fine-Grained Systems*. 2. ed. Sebastopol: O'Reilly Media, 2021.

[2] CERNY, T.; BUSHEY, R.; TCHAKOUNTÉ, F. et al. On Microservices Architecture: A Systematic Mapping Study. *The Journal of Systems and Software*, v. 142, p. 72-85,

2018. DOI: 10.1016/j.jss.2018.04.058.

[3] SHAFIEI, H.; KHONSARI, A.; MOUSAVI, P. Serverless Computing: A Survey of Opportunities, Challenges and Applications. *arXiv*, 2019. Disponível em: https://arxiv.org/abs/1911.01296. Acesso em: 17 set. 2026.

[4] JOHN, J.; GUPTA, S. A Survey on Serverless Computing. *arXiv*, 2021. Disponível em: https://arxiv.org/abs/2106.11773. Acesso em: 17 set. 2026.

[5] LI, Z. et al. The Serverless Computing Survey: A Technical Primer for Design Architecture. *arXiv*, 2021. Disponível em: https://arxiv.org/abs/2112.12921. Acesso em: 17 set. 2026.

[6] WEN, J. et al. Rise of the Planet of Serverless Computing: A Systematic Review. *arXiv*, 2022. Disponível em: https://arxiv.org/abs/2206.12275. Acesso em: 17 set. 2026.

[7] BROOKER, M. et al. On-demand Container Loading in AWS Lambda. *arXiv*, 2023. Disponível em: https://arxiv.org/abs/2305.13162. Acesso em: 17 set. 2026.

[8] INDRASIRI, K.; SIRIWARDENA, P. *Microservices Security in Action*. Shelter Island: Manning Publications, 2020.

[9] AWS. AWS Well-Architected Framework. Disponível em: https://docs.aws.amazon.com/wellarchitected/. Acesso em: 17 set. 2026.

[10] AWS. Amazon API Gateway Developer Guide. Disponível em: https://docs.aws.amazon.com/apigateway/. Acesso em: 17 set. 2026.

[11] AWS. AWS Lambda Developer Guide. Disponível em: https://docs.aws.amazon.com/lambda/. Acesso em: 17 set. 2026.

[12] AWS. Amazon Cognito Developer Guide. Disponível em: https://docs.aws.amazon.com/cognito/. Acesso em: 17 set. 2026.

[13] AWS. Amazon CloudFront Developer Guide. Disponível em: https://docs.aws.amazon.com/cloudfront/. Acesso em: 17 set. 2026.

[14] SILVA, L. M. L. da; SOUZA, A. C. A.; TODESCO, G.; SILVA, C. A.; SILVA, G. C. da; TAKATU, D. S. Integração entre a Metodologia SENAI de Educação Profissional (MSEP) e a inteligência artificial no ensino de engenharia de software: um modelo conceitual. *Revista Científica SENAI-SP - Educação, Tecnologia e Inovação*, v. 4, n. 1, p. 139-157, 2026. DOI: 10.62899/rcs.v4i1.266. Acesso em: 23 set. 2026.

[15] ADZIC, G.; CHATLEY, R. Serverless computing: economic and architectural impact. In: *ESEC/FSE 2017*. New York: ACM, 2017. p. 884-889. DOI: 10.1145/3106237.3117767.

AGRADECIMENTOS

Agradece-se aos professores Deivison Shindi Takatu e Glauco Todesco pela orientação e pelas contribuições ao desenvolvimento desta pesquisa.

SOBRE OS AUTORES

Gabriel Ribeiro de Camargo Miranda é discente de Engenharia de Software e desenvolve pesquisa na área de arquitetura de software e computação em nuvem. Deivison Shindi Takatu e Glauco Todesco participam como orientadores da pesquisa. As informações institucionais e os contatos dos autores devem ser confirmados antes da submissão na plataforma do evento.